Linguaggi di programmazione 1

Anno 2021

Valentino Bocchetti

Contents

1 Introduzione al corso e informazioni **2**

1.1 Esame 2

2 Suddivisione del corso **2**

2.1 Parte I 2

2.2 Parte II (modello object-oriented, Java) 2

2.3 Parte III (linguaggi funzionali, ML) 3

2.4 Parte IV (linguaggi logici, Prolog) 3

3 Lezione del 09-03 **4**

3.1 Breve storia 4

3.2 Definizione di base 5

3.3 Linguaggi completi 5

3.4 Macchine astratte 5

3.5 Traduttori del linguaggio 8

3.6 SRT 10

3.7 Compilazione ed esecuzione 10

3.8 Proprietà dei linguaggi 12

3.9 Criteri di scelta dei linguaggi 12

1 Introduzione al corso e informazioni

`Email` → pab@unina.it

`Ricevimento` → da richiedere in precedenza con una mail

1.1 Esame

- Uno scritto java + altre parti del corso
- In caso di dubbi c'è un orale

Lo scritto può essere mantenuto per la durata di un anno

2 Suddivisione del corso

2.1 Parte I

- Storia e concetti di base
- Primi cenni ai paradigmi dei linguaggi di programmazione
- Il modello imperativo

2.2 Parte II (modello object-oriented, Java)

- Studio dei costrutti più avanzati
  - qualificatori di classi
  - metodi e attributi
  - classi astratte
  - interfacce
  - template
  - classi interne
  - gestione degli errori
  - eccezioni

2.3 Parte III (linguaggi funzionali, ML)

- ML e costrutti avanzati
- funzioni di ordine superiore
- polimorfismo
- tipi di dato astratti e interfacce
- template
- gestione degli errori
- eccezioni
- type inference

2.4 Parte IV (linguaggi logici, Prolog)

- Costrutti fondamentali
  - termini
  - predicati
  - regole
- Tecniche di programmazione avanzate
  - invertibilità
  - nondeterminismo

3 Lezione del 09-03

3.1 Breve storia

Il primo linguaggio introdotto è il FORTRAN (introdotto con lo scopo di calcoli scientifici). Nasce la concezione di variabile

Anno	Linguaggio
1954	FORTRAN (FORmula TRANslation)
1960	COBOL
	ALGOL (60 e 68)
	`PL/1`
	Simula 67
	PASCAL
	LISP
	APL
	BASIC
`1970/1980`	PROLOG
	SMALLTALK
	C
	`MODULA/2`
	ADA

- **Cobol** → orientato ai database (introduce i record).
- **Agol60** → indipendenza dalla macchina e definizione mediante grammatica (supporto dell'iterazione e ricorsione, introduzione delle strutture a blocco).
- **Lisp** → primo vero linguaggio di manipolazione simbolica, linguaggio funzionale (non esiste lo statement di assegnazione).
- **Prolog** → primo (e principale) linguaggio di programmazione logica, porta l'invertibilità e una programmazione in stile nondeterministico (essenzialmente non tipato; queste vengono ottenute mediante metaprogrammazione).
- **Simula 67** → classe come incapsulamento di dati e procedure, istanze delle classi (oggetti).
- `PL/1` abilità ad eseguire procedure specificate quando si verifica una condizione eccezionale; (multitasking).
- **Pascal** → programmazione strutturata, tipi di dato definiti da utente (ricchezza di strutture dati); nessuna encapsulation.

3.2 Definizione di base

Linguaggio di programmazione

È un linguaggio che è usato per esprimere (mediante un programma) un processo con il quale un processore può risolvere un problema.

Processore

È la macchina che eseguirà il processo descritto dal programma (da non intendere come un singolo oggetto, ma coome una *architettura di elaborazione*).

Programma

È l'espressione codificata di un processo.

3.3 Linguaggi completi

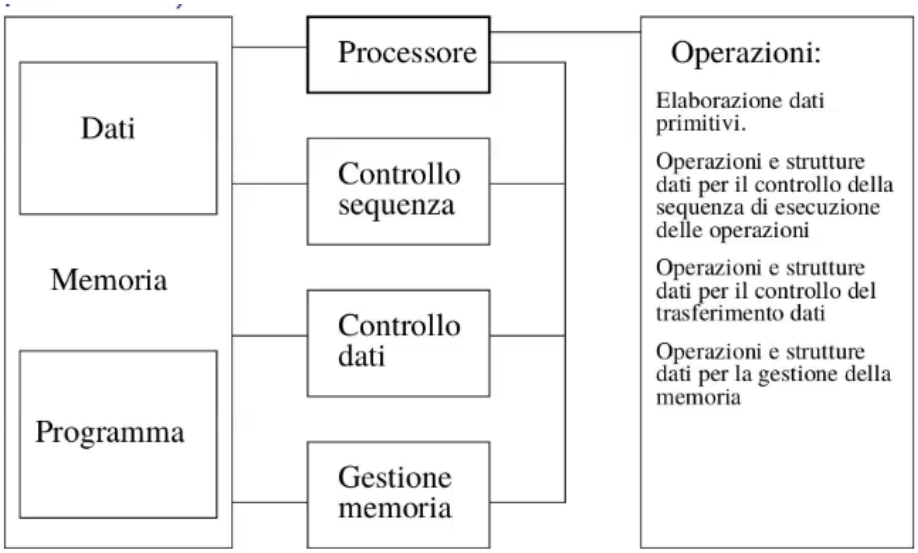
Sono linguaggi in graddo di programmare qualunque funzione calcolabile (*general purpose*). Tecnicamente devono poter simulare qualunque *macchina di Turing.*

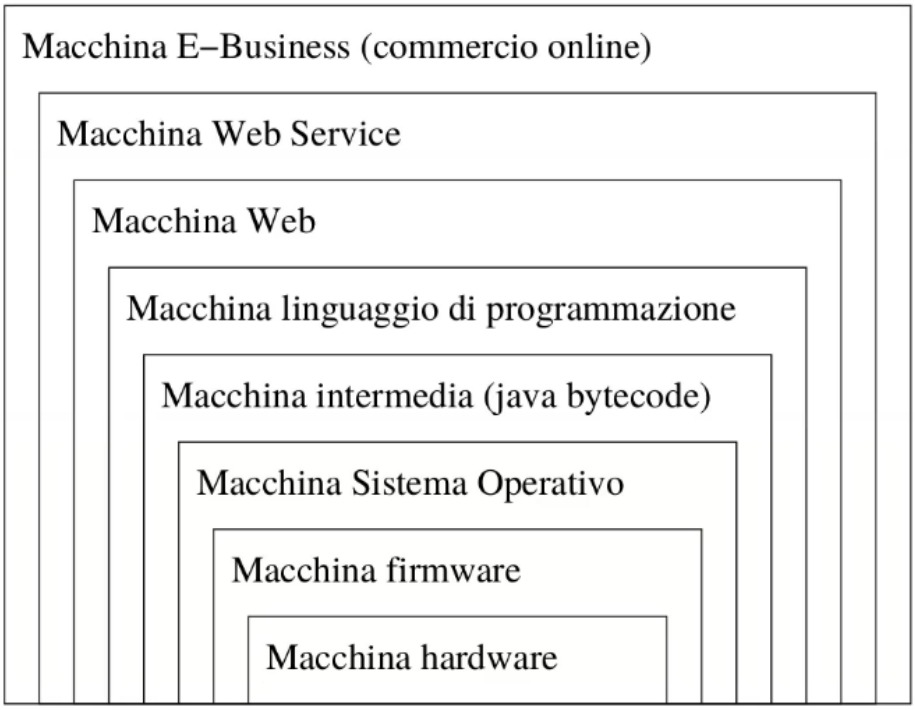
Sono completi solo quelli che riescono ad esprimere anche programmi di cui non è decidibile la terminazione. Per dimostrare la completezza di un linguaggio lo si usa per simulare arbitrarie macchine di Turing.

Ad esempio non sono non è un linguaggi completi **SQL** (perchè la terminazione dei programmi è sempre decidibile; è spesso immerso in linguaggio completo) e **HTML**.

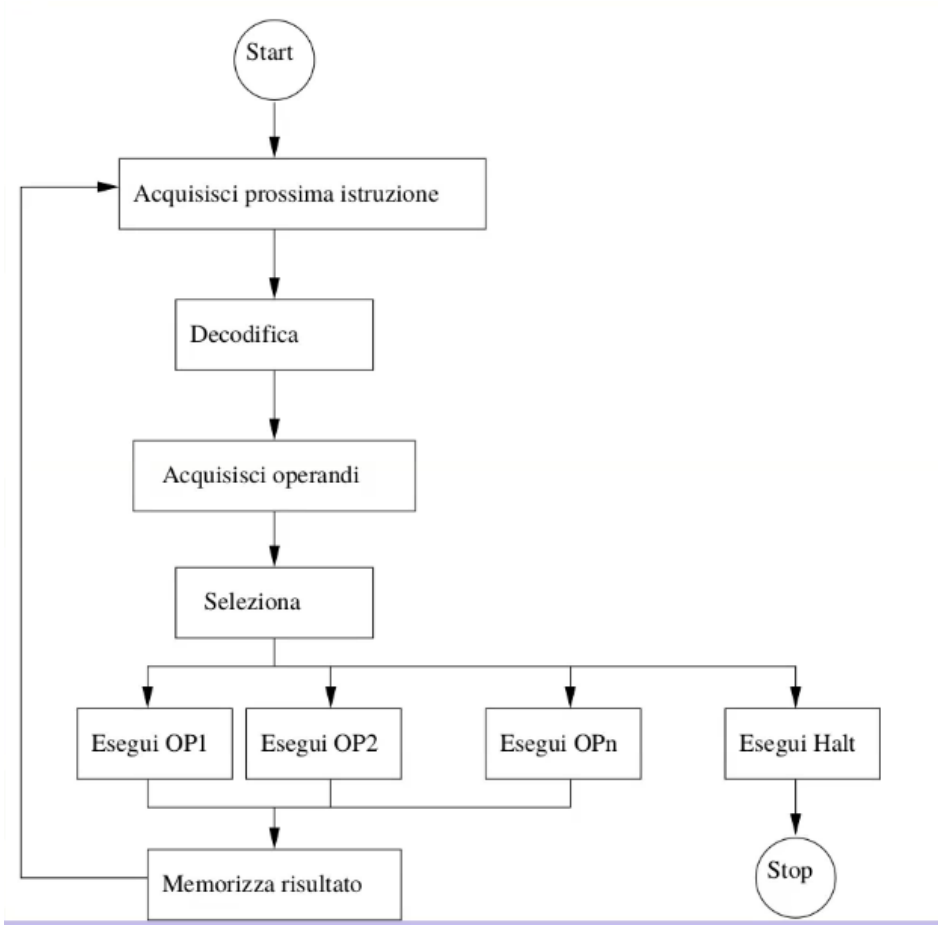
3.4 Macchine astratte

Dato un linguaggio di programmazione L, una macchina astratta per L (in simboli M_L) è un qualsiasi insieme di struttre dati e algoritimi che permettano di memorizzare ed eseguire programmi scritti in L. La struttura di una macchina astratta è (essenzialmente memoria e processore):


Processore
Dati
Memoria
Programma
Controllo sequenza
Controllo dati
Gestione memoria
Operazioni:
Elaborazione dati primitivi.
Operazioni e strutture dati per il controllo della sequenza di esecuzione delle operazioni
Operazioni e strutture dati per il controllo del trasferimento dati
Operazioni e strutture dati per la gestione della memoria


Macchina E−Business (commercio online)
Macchina Web Service
Macchina Web
Macchina linguaggio di programmazione
Macchina intermedia (java bytecode)
Macchina Sistema Operativo
Macchina firmware
Macchina hardware

Processore della macchina astratta



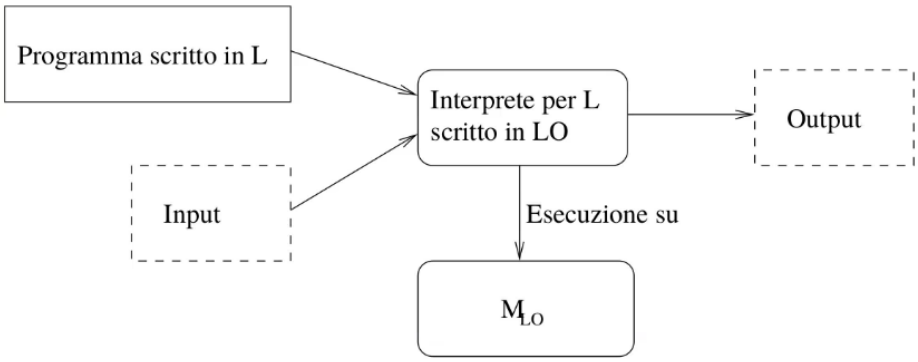
Nel tempo si è pensato a vari modi di implementarlo:

- Hardware (es per *Lisp* e *Prolog*), ma abbandonato rapidamente;
- Firmware → soluzione più flessibile e semplice (ed economico);
- Software → come nel caso della jvm;

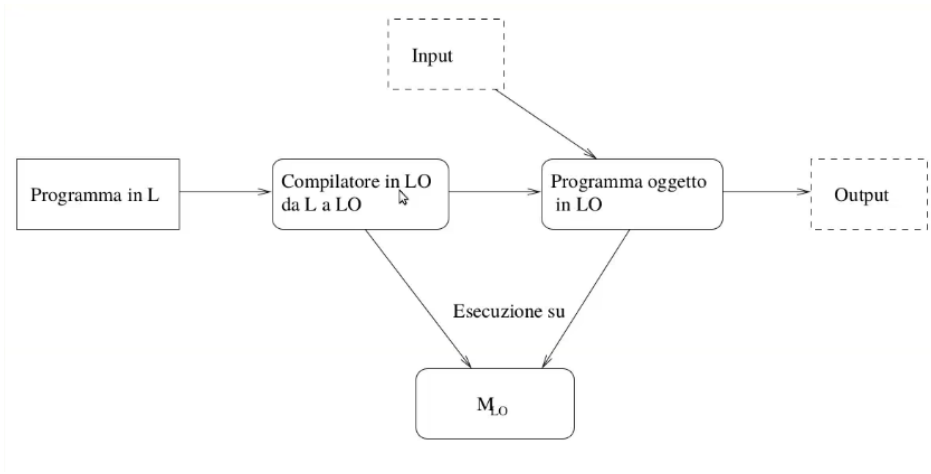
3.5 Traduttori del linguaggio

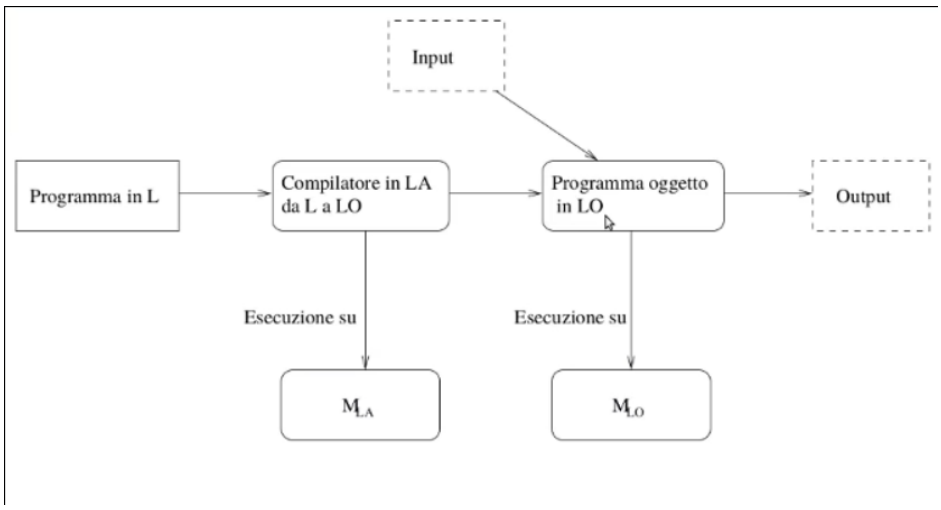
- **Interpreti** → traducono ed eseguono un costrutto alla volta
  - PRO : debug
- **Compilatori** → prima traducono l'intero programma, che poi può essere eseguito
  - PRO : velocità di esecuzione finale
  - PRO : più controlli e in anticipo

Interpretazione pura



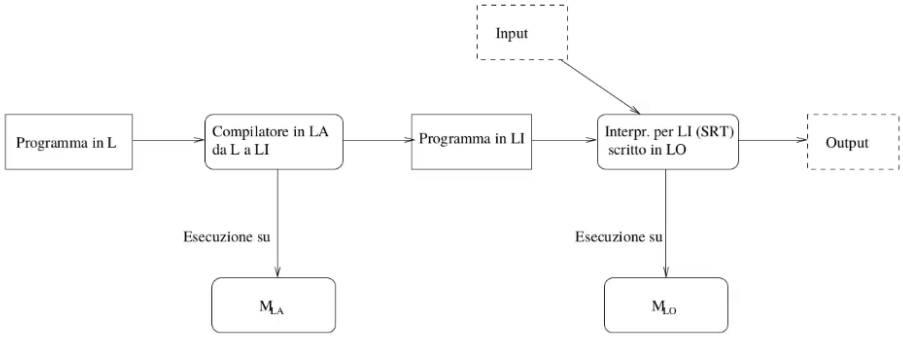
Compilazione pura (caso semplice e generale)





Nella compilazione pura può accadere che L ed LO siano uguali (il vantaggio è il riutilizzo del compilatore $\rightarrow$ **custom compilation**)

Compilazione per macchina intermedia



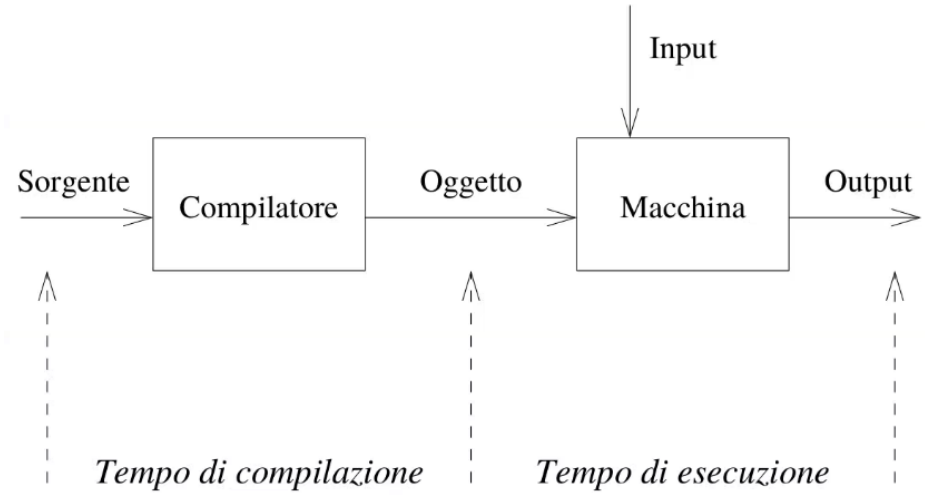
($SRT \rightarrow$ supporto al run time) Torna utile nel caso di linguaggi molto diversi dalla macchina astratta sottostante. Invece di creare hardware apposito creiamo un interprete che lo sostituisca

3.6 SRT

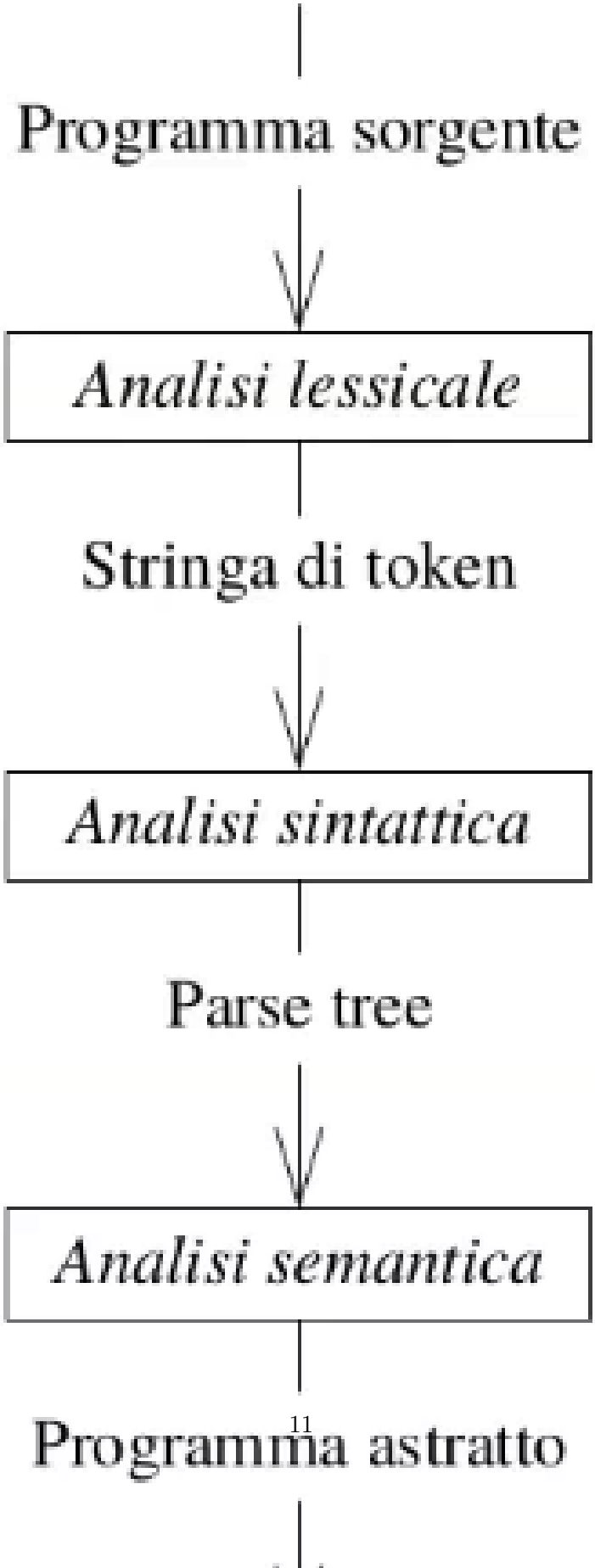
- Funzionalità aggiuntive (rispetto alla macchina astratta sottostante)
- Funzioni di basso livello / interfacce con il S.O. (es `I/O`)
- Gestione della memoria;
- Garbage collection;
- Gestione dell'heap
- Gestione dello stack

Non è necessariamente una macchina astratta radicalmente diversa, ma a volte solo un pacchetto di funzioni o librerie aggiunte automaticamente al codice oggetto

3.7 Compilazione ed esecuzione



Fasi della compilazione


Programma sorgente
Analisi lessicale
Stringa di token
Analisi sintattica
Parse tree
Analisi semantica
Programma astratto

(Analisi lessicale e sintattica sono stati un problema → venivano fatte in maniera artigianale)

3.8 Proprietà dei linguaggi

- **Semplicità** → (coincisione) VS (leggibilità)
  - **Semantica** → minimo numero di concetti e strutture
  - **Sintattica** → unica rappresentabilità di ogni concetto
- **Astrazione** → rappresentare solo attributi essenziali
  - **Dati** → nascondere i dettagli di oggetti
  - **Procedure** → facilitare la modularità del progetto
- **Espressività** → (facilità di rappresentazione di oggetti) VS (semplicità)
- **Ortogonalità** → meno eccezioni alla regole del linguaggio
- **Portabilità**

3.9 Criteri di scelta dei linguaggi

- Disponibilità dei traduttori;
- Maggiore conoscenza da parte del programmatore;
- Esistenza di standard di portabilità (es ADA);
- Comodità dell'ambiente di programmazione;
- Sintassi aderente al problema;
- Semantica aderente alla architettura fisica non importante (non è più un criterio stringente).